Deviations from Submitted Specification

Webhook Aggregator System (wagg / WACDP)

Project Code: BPS49 — Webhook Aggregator Core Development Pipeline

Team: Sumedh Girish (`PES1UG24CS480`), Subramani B M (`PES1UG24CS473`), Sujan S Halanannavar (`PES1UG24CS477`), Skanda Shyam Nadig (`PES1UG24CS458`)

Baseline documents: SRS v1.0, Software Test Plan v1.0, SADD v1.0 (all dated 29 Sept 2026)

Implementation compared: `main` @ `aa5baf6` (`src/`, ~900 lines of Rust)

---

1. Purpose

This document records every place where the delivered implementation differs from the Part-1 documents (SRS, Test Plan, SADD), explains why, and states the status of each deviation. It is the reference for the deviation discussion in the final demo.

Each deviation has one of three statuses:

Status	Meaning
Changed	Implemented, but with a different design from the one specified. The behaviour the requirement exists for is still delivered.
Partial	Part of the requirement is implemented; the remainder is listed.
Not implemented	Not present in the code. Carried over as open backlog work.

---

2. Summary

ID	Area	Specified	Implemented	Status
D-01	Language & toolchain	C++20, Clang/GCC, OpenSSL	Rust 2024, Tokio, Axum, Reqwest	Changed
D-02	Buffering / persistence	In-memory `service_queue` + `dispatch_queue`, SQLite/PostgreSQL for DLQ only	Every request persisted in a MariaDB `queries` table; the table *is* the queue	Changed
D-03	Concurrency model	Elastic worker thread pool (4–64 threads) + dispatcher thread	One async "claimer" task per downstream service, bounded by a per-service semaphore	Changed

ID	Area	Specified	Implemented	Status
D-04	Ingestion API	`POST /api/v1/webhooks/ingest`, Bearer token, `202 Accepted` + `task_id`	`POST /webhook/{endpoint}`, Ed25519-signed body, `200 OK` with no body	Changed
D-05	Routing model	Caller supplies `target_url` per request	Endpoint name maps to a registered service (`routes` → `services`)	Changed
D-06	Ingress authentication	Bearer token (SEC-REQ-02)	Per-client Ed25519 signature over SHA-256 of the request	Changed
D-07	Idempotency / deduplication	Not specified at ingress	Unique `(client_id, event_id, service_id)`; duplicates ignored	Added
D-08	Retry policy	1 + 5 retries, delays 1 s, 2 s, 4 s, 8 s, 16 s, 5 s timeout	5 attempts total, delays 0.5 s, 1 s, 2 s, 4 s (cap 30 s), 10 s timeout	Changed
D-09	Dead-letter storage	Separate `dead_letter_webhooks` table + append-only audit log	Row marked `FAILED` in `queries`; failure reason only in process log	Partial
D-10	Crash recovery	Reload recurring schedules; DLQ intact	`PENDING` / `PROCESSING` rows re-claimed on start-up and after DB errors	Changed
D-11	Recurring schedules (FR-05, FR-06)	Cron/interval timers, `recurring_schedules` table	—	Not implemented
D-12	Status & health API (FR-10)	`GET /api/v1/webhooks/{id}/status`, `GET /api/v1/health`	—	Not implemented
D-13	DLQ inspection & redrive (FR-11)	`GET /api/v1/webhooks/failed`, `POST /api/v1/webhooks/{id}/retry`	—	Not implemented
D-14	Egress signing (FR-04, SEC-REQ-01)	HMAC-SHA256 `X-Webhook-Signature`, `X-Webhook-ID`, `X-Webhook-Timestamp`	Only `Content-Type: application/json` sent	Not implemented

ID	Area	Specified	Implemented	Status
D-15	Rate limiting / backpressure (SEC-REQ-02, NFR-07)	Token bucket, HTTP 429 + `Retry-After` at queue limit	—	Not implemented
D-16	TLS 1.3 only (SEC-REQ-03)	TLS 1.3 enforced inbound and outbound	Plain HTTP listener on port 31337; Reqwest defaults outbound	Not implemented
D-17	Log sanitisation (SEC-REQ-04)	Recursive redaction of secrets in audit log	—	Not implemented
D-18	Payload limits (FR-01)	413 above 64 KB, staging above 2 KB	Axum default body limit (2 MB); no staging	Changed
D-19	Replay protection (Test Plan §5.1.2)	Timestamp window check	Timestamp is signed but not checked against the clock	Partial
D-20	Automated tests (Test Plan §5, §10)	Catch2/PyTest suites, TSan, Valgrind, k6	CI build + MariaDB smoke test; no unit or integration tests	Partial

---

3. Detailed Deviations

D-01 — Implementation language changed from C++20 to Rust

• **Specified:** SRS NFR-10, NFR-11, §3.5 ("Zero Raw Pointers", Valgrind/ASan); Test Plan §6.2 (Clang 16+/GCC 12+, OpenSSL 3.0, TSan, Catch2).

• **Implemented:** Rust 2024 edition (`Cargo.toml`, crate `wagg`) using Tokio, Axum, Reqwest, `mysql_async` and `ed25519-dalek`.

• **Rationale:** The Rust compiler enforces the goals the SRS set for C++: memory safety without raw pointers (§3.5), no data races (NFR-09) and leak-free ownership. Requirements we had planned to verify with Valgrind, ASan and TSan are instead guaranteed when the code compiles. Tokio and Axum also provide the async I/O and HTTP stack that we would have had to assemble by hand in C++.

• **Impact:** NFR-10 (modularity) and NFR-11 (portability) still hold: the code is split into `listener`, `dispatcher` and `db` modules and builds on Linux (CI runs on `ubuntu-latest`) and macOS. The tools named in Test Plan §6.2 no longer apply; see D-20.

D-02 — Durable database-backed queue instead of in-memory queues

• **Specified:** SRS FR-02, FR-04, FR-07; SADD components 2 and 5 (`service_queue`, `dispatch_queue`), §12.1; SRS §3.4 (SQLite/PostgreSQL for DLQ and schedules only).

• **Implemented:** `src/db/mod.rs` creates four MariaDB tables: `clients`, `services`, `routes` and `queries`. An accepted request is written to `queries` with status `PENDING` before the HTTP response is sent. Dispatchers read work from that table (`PENDING → PROCESSING → SUCCESS | FAILED`).

- **Rationale:** NFR-04 requires zero silent data loss. With in-memory queues, a crash loses every accepted request that hasn't been delivered yet. Writing to the database before acknowledging means an accepted request survives a restart. The `work (service_id, status, seq)` index keeps claiming efficient.
- **Impact:** The queue capacities (50,000 default / 100,000 burst, NFR-07) and the ~200-byte task descriptors no longer apply; the queue is bounded by database storage instead. Each request now costs one database round trip, so the latency targets in NFR-01 and NFR-02 must be re-measured against this design.

D-03 — Per-service claimers instead of a shared worker thread pool

- **Specified:** SRS FR-03, NFR-08 (4–64 elastic worker threads); SADD §4.3, components 3 and 6.
- **Implemented:** `src/dispatcher/mod.rs`. One async *claimer* task runs per downstream service. It claims up to `capacity` `PENDING` rows at a time (`services.capacity`, default 10) and starts one delivery task per row. A per-service semaphore means a service never has more than `capacity` requests in flight. The listener "rings a doorbell" (`tokio::sync::Notify`) to wake the right claimer; a claimer also re-checks every 5 s while idle.
- **Rationale:** Concurrency is limited per destination, so one slow or failing service can't use up all the workers and starve the others. Because each service's rows have exactly one owner, claiming needs no row locks (see the comment on `Database::claim`). Async tasks replace OS threads, so the 4–64 thread range doesn't apply.
- **Impact:** FR-03 categorisation by `type` doesn't exist because there is only one request type (see D-11).

D-04 — Ingestion endpoint, request format and response changed

- **Specified:** SRS §3.1.1, FR-01; SADD §10.1. The specified request is `POST /api/v1/webhooks/ingest` with body `{type, target_url, payload, schedule}`, answered with `202 Accepted` and `{task_id, status, timestamp}`.
- **Implemented:** `src/listener/mod.rs` and `src/listener/request.rs`. The endpoint is `POST /webhook/{endpoint}` on `0.0.0.0:31337`, with this body:

```
{
  "client_name": "string",
  "nonce":       "32 hex chars (16-byte event id)",
  "id":          "128 hex chars (64-byte Ed25519 signature)",
  "datetime":    1760000000,
  "payload":     "exact JSON text that was signed"
}
```

The current responses are:

Situation	Code
Accepted, or duplicate ignored	`200 OK`
Malformed hex or timestamp	`400 Bad Request`
Body is not valid JSON, or fields are missing or the wrong type	`400`, `415` or `422` (Axum defaults)
Endpoint has no route	`400 Bad Request`
Unknown client, or signature fails	`401 Unauthorized`
Target service marked `DOWN`	`503 Service Unavailable`

Situation	Code
Database error, including a `payload` that is not valid JSON	`500 Internal Server Error`

- **Rationale:** Follows from D-05 and D-06. The payload is a string so that the server verifies exactly the bytes the client signed.
- **Impact:** The response has no `task_id`, so a caller can't look up a request later (see D-12). `200` is returned where the spec said `202`. A payload that isn't valid JSON returns 500 rather than 400. **Follow-up:** return `202` with the `seq` as the task id, and validate the payload JSON in the listener.

D-05 — Destinations are registered services, not per-request URLs

- **Specified:** `target_url` supplied in every request (FR-01).
- **Implemented:** The caller names an *endpoint*. The `routes` table maps it to a row in `services` (`url`, `status UP/DOWN`, `capacity`).
- **Rationale:** If callers could choose any URL, the aggregator could be used to send requests to arbitrary hosts (SSRF). Registering services also gives each destination its own capacity limit and an UP/DOWN switch, which D-03 relies on.
- **Impact:** Operators must register services and routes in the database before use; there is no management API for this yet.

D-06 — Ed25519 request signatures instead of Bearer tokens

- **Specified:** SEC-REQ-02 (Bearer token in `Authorization`); Test Plan §5.1.3.
- **Implemented:** Each client has a registered 32-byte Ed25519 public key (`clients.public_key`). The server computes `SHA-256(client_name ■ event_id ■ datetime(be i64) ■ payload)` and verifies the signature against it with `verify_strict` (`src/db/query.rs`).
- **Rationale:** This is stronger than SEC-OBJ-02 requires. The server stores only public keys, so a leaked database can't be used to impersonate clients. Each request is tied to one client and can't be altered without breaking the signature, which a Bearer token can't guarantee.
- **Impact:** Test Plan §5.1.3 and TC-SEC-03 need to be rewritten for signature checks (wrong key, modified payload, modified timestamp) rather than tokens.

D-07 — Ingress deduplication added

- **Specified:** Not specified at ingress; the SADD covers idempotency only for egress.
- **Implemented:** `UNIQUE KEY dedup (client_id, event_id, service_id)` on `queries`, written with `INSERT … ON DUPLICATE KEY UPDATE`. A repeated event is logged and not dispatched again.
- **Rationale:** Callers can safely retry after a network failure, and each event is still delivered at most once per service.

D-08 — Retry parameters differ

- **Specified:** FR-07, FR-08 and NFR-06 specify 1 attempt plus 5 retries (6 in total), waits of 1 s, 2 s, 4 s, 8 s and 16 s (± 100 ms), a 5,000 ms timeout, and 400/401/403/404 treated as non-retryable.
- **Implemented:** `src/dispatcher/mod.rs`:
- **Attempts:** `MAX_ATTEMPTS = 5` in total.
- **Waits:** `BASE_DELAY = 500 ms`, doubling each time and capped at `MAX_DELAY = 30 s`, so the waits are 0.5 s, 1 s, 2 s and 4 s.

- **Timeout:** `ATTEMPT_TIMEOUT = 10 s` per attempt.
- **Retried:** 5xx responses, 429, timeouts and connection errors.
- **Not retried:** all other non-2xx responses and request-builder errors give up immediately.
- **Rationale:** The classification is the one the SRS specifies, extended to *all* 4xx codes rather than just four. A shorter first wait lets a brief outage clear sooner. The delay database calls use when retrying (`persist`) is capped separately at 5 s.
- **Impact:** TC-FR-08 and TC-NFR-06 expect the old timings. **Decision for the demo:** either set the constants to `1 s` base, 6 attempts and a `5 s` timeout (a three-line change), or update SRS FR-08 and NFR-06 to the values above.

D-09 — Dead-letter storage is a status, not a table

- **Specified:** FR-09, SRS §3.4.1 (`dead_letter_webhooks` with attempt count, last error and failure time), an append-only JSON audit log, and SADD §11.3.
- **Implemented:** A request that fails permanently is marked `FAILED` in `queries` and keeps its payload, client, service and received time. The reason for the failure is written to the process log (`pretty_env_logger`) and **not** stored in the database.
- **Status:** Partial. The *zero silent data loss* goal (NFR-04) is met, because nothing is deleted and every failure can be found by status. The attempt count, the last error and a separate append-only audit log are missing.

D-10 — Crash recovery covers queued work, not schedules

- **Specified:** SRS §3.6.2 (reload `recurring_schedules` within 3 s).
- **Implemented:** On start-up, `Dispatcher::run` finds every service with `PENDING` rows and wakes its claimer. After a failed claim, the claimer first adopts any `PROCESSING` rows it does not already own, in case the claim's `UPDATE` committed without the reply arriving. Shutdown on Ctrl-C lets in-flight deliveries finish and store their final status first.
- **Rationale:** With the database-backed queue (D-02), accepted work is what has to survive a crash. Schedule reload doesn't apply because schedules don't exist (D-11).
- **Known gap:** If the process is killed while rows are `PROCESSING` and no claim error happens afterwards, those rows are not picked up again on the next start, because only `PENDING` services are scanned. **Follow-up:** reset `PROCESSING` to `PENDING` at start-up.

D-11 — Recurring schedules not implemented (FR-05, FR-06, NFR-03)

- **Specified:** Cron or interval schedules, a `scheduler_mapping` of timer handles, the `recurring_schedules` table, ± 10 ms precision, and a reload on restart.
- **Implemented:** Not implemented. Every request is one-shot.
- **Backlog:** Issue #10 (`WACDP-6`, Sprint-2) remains open.

D-12 — Status and health API not implemented (FR-10)

- **Specified:** `GET /api/v1/webhooks/{task_id}/status` and `GET /api/v1/health`.
- **Implemented:** Not implemented. The `queries.status` column holds everything a status endpoint would need, so this is a thin layer on the existing schema.
- **Backlog:** Issue #11 (`WACDP-8`, Sprint-2).

D-13 — DLQ inspection and manual redrive not implemented (FR-11)

- **Specified:** `GET /api/v1/webhooks/failed` and `POST /api/v1/webhooks/{task_id}/retry`.
- **Implemented:** Not implemented. A row can be redriven by hand with `UPDATE queries SET status='PENDING' WHERE seq=…`; the claimer picks it up within 5 s.
- **Backlog:** Issue #11.

D-14 — Outgoing webhooks are not signed (FR-04, SEC-REQ-01)

- **Specified:** An HMAC-SHA256 `X-Webhook-Signature` header (`t=…,v1=…`), plus `X-Webhook-ID` and `X-Webhook-Timestamp`.
- **Implemented:** Only `Content-Type: application/json` and the stored payload are sent.
- **Impact:** Receivers can't verify that a webhook came from wagg, so SEC-OBJ-01 is **not met**. TC-SEC-01 and Test Plan §5.1.1 will fail.
- **Backlog:** Issue #4 (`WACDP-4C`). Its title already reflects the Ed25519 ingress design.

D-15 — No rate limiting or HTTP 429 backpressure (SEC-REQ-02, NFR-07)

- **Specified:** A per-client token bucket, and `429 Too Many Requests` with `Retry-After: 5` at queue capacity.
- **Implemented:** Not implemented at ingress. Load on destinations *is* limited, by the per-service `capacity` (D-03).
- **Impact:** SEC-OBJ-03 is only partly met. TC-SEC-04 and Test Plan §5.1.4 will fail.

D-16 — TLS not enforced (SEC-REQ-03)

- **Specified:** TLS 1.3 only, inbound and outbound.
- **Implemented:** The listener serves plain HTTP on port 31337. Outbound calls use Reqwest's default client, which uses TLS for `https://` URLs but does not enforce version 1.3 and also allows `http://`.
- **Rationale:** We intend to terminate TLS at a reverse proxy in front of the service; this is not documented in the SADD deployment view (§7.1) yet.

D-17 — No log sanitisation (SEC-REQ-04)

- **Specified:** Recursive redaction of sensitive keys before audit logging.
- **Implemented:** Payloads are not written to the logs. Logs contain URLs, client and service names, event ids and error text. There is no redaction step.
- **Backlog:** Issue #12 (`WACDP-11`).

D-18 — Payload size limits differ

- **Specified:** `413` above 64 KB; payloads above 2 KB staged to temporary storage.
- **Implemented:** Axum's default 2 MB request-body limit applies. The payload is stored in a MariaDB `JSON` column, so no staging is needed.

D-19 — No replay window

- **Specified:** Test Plan §5.1.2 (reject requests whose timestamp is outside the allowed window).
- **Implemented:** The timestamp is covered by the signature, and resending a captured request is dropped as a duplicate (D-07). However, the timestamp is never compared with the server clock, so a validly signed request that was never delivered is accepted no matter how old it is.
- **Follow-up:** Reject requests where `|now − datetime|` is more than 300 s.

D-20 — Automated test coverage

• **Specified:** Test Plan §5.2–§5.5 and §10 (28 test cases), using Catch2/PyTest, TSan, Valgrind and k6/wrk.

• **Implemented:** The GitHub Actions workflow (`.github/workflows/ci.yml`) runs `cargo check`, `cargo build --release` and `cargo test`, then a smoke test that starts the listener against a MariaDB service container and checks the schema is created. There are **no** `#[test]` functions in the crate yet, so `cargo test` runs zero tests. Performance figures are in `docs/4_Performance_Benchmark_Datasheet_Report.pdf`.

• **Impact:** Most cases in Test Plan §10 have to be run by hand with `curl` and a mock receiver. The cases tied to D-11 to D-17 can't pass yet.

---

4. Effect on Requirements Traceability

Requirement	Status in the delivered build
FR-01 Ingestion & validation	Met, with a different interface (D-04, D-05, D-18)
FR-02 Non-blocking enqueue	Met by a durable table instead of an in-memory buffer (D-02)
FR-03 Categorisation	Not applicable: one-shot only (D-11)
FR-04 One-shot execution	Partial: no egress signing (D-14)
FR-05 / FR-06 Recurring schedules	Not met (D-11)
FR-07 Dispatch	Met; 10 s timeout, TLS not enforced (D-08, D-16)
FR-08 Exponential backoff	Met, with different constants (D-08)
FR-09 Dead-letter persistence	Partial (D-09)
FR-10 Status & health API	Not met (D-12)
FR-11 Redrive API	Not met (D-13)
NFR-04 Zero silent data loss	Met (D-02, D-10), apart from the `PROCESSING` gap noted in D-10
NFR-07 Bounded memory	Met per service (D-03); no ingress 429 (D-15)
NFR-09 Thread safety	Guaranteed by the Rust compiler (D-01)
NFR-10 / NFR-11 Modularity, portability	Met, in Rust (D-01)
SEC-REQ-01 Egress HMAC	Not met (D-14)
SEC-REQ-02 Ingress auth & rate limiting	Authentication exceeds the spec (D-06); rate limiting not met (D-15)
SEC-REQ-03 TLS 1.3	Not met in-process (D-16)
SEC-REQ-04 Audit log & sanitisation	Not met (D-09, D-17)

---

5. Follow-up Actions

#	Action	Closes
1	Update SRS §3.1, FR-01, §3.4 and SEC-REQ-02 to describe the Ed25519 / registered-service design, or keep this document as the formal change record	D-04 – D-07
2	Align the retry constants with FR-08, or amend FR-08 and NFR-06	D-08
3	Return `202` with a task id; validate payload JSON in the listener	D-04
4	Reset `PROCESSING` rows to `PENDING` at start-up	D-10
5	Add a replay-window check	D-19
6	Implement egress HMAC headers	D-14
7	Add the status, health and redrive endpoints	D-12, D-13
8	Add `attempts` and `last_error` columns, and an append-only audit log	D-09, D-17
9	Add unit tests (signature verification, retry classification) and an integration test with a mock receiver	D-20
10	Recurring schedules	D-11